

Experiment 07

Aim: Perform Clustering Analysis using **Scipy** and **Scikit-learn**.

Problem Statement:

1. **Perform Clustering (K-Means and DBScan)** to determine the relationship between variables.
2. **Apply a Clustering Model** technique to predict the data based on the given dataset.

Dataset Description:

The dataset contains more than **1 lakh instances**, fulfilling the requirement for **Big Data analysis**. The following are the key features used in the regression models:

- **Cycle_Index**: Index of the cycle during battery charging/discharging.
- **Discharge Time (s)**: Time taken for battery discharge.
- **Decrement 3.6-3.4V (s)**: Time decrements between specific voltage levels.
- **Max. Voltage Discharge (V)**: Maximum voltage during discharge.
- **Min. Voltage Charge (V)**: Minimum voltage during charge.
- **Time at 4.15V (s)**: Time spent at a specific voltage level.
- **Time constant current (s)**: Duration of constant current phase.
- **Charging time (s)**: Total time taken for charging.
- **RUL (Remaining Useful Life)**: Predicted remaining life of the battery.

The dataset is preprocessed to remove missing values and scale numerical features before regression analysis.

Theory:

1]K-Means Clustering

K-Means is a centroid-based clustering algorithm that partitions data into K clusters based on the minimization of intra-cluster variance.

Mathematical Formulation:

Given a dataset $X=\{x_1, x_2, \dots, x_n\}$, where x_i is a d-dimensional data point, the objective of K-Means is to minimize the following cost function:

$$J = \sum_{i=1}^K \sum_{x_j \in C_i} ||x_j - \mu_i||^2$$

where:

- K is the number of clusters.
- C_i represents the set of points assigned to cluster i .
- μ_i is the centroid of cluster i .
- $||x_j - \mu_i||^2$ is the squared Euclidean distance.

Algorithm Steps

1. **Initialization:** Randomly select K initial cluster centroids $\mu_1, \mu_2, \dots, \mu_K$.
2. **Assignment Step:** Assign each data point to the closest centroid using:

$$C_i = \{x_j \mid ||x_j - \mu_i||^2 \leq ||x_j - \mu_k||^2, \forall k \neq i\}$$

3. **Update Step:** Compute new centroids as the mean of points in each cluster:

$$\mu_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

4. **Repeat Steps 2 and 3** until convergence (i.e., centroids do not change significantly).

Limitations

1. Sensitive to initial centroid selection.
2. Works best for spherical, equally sized clusters.
3. Poor performance on non-linearly separable data.

2]DBSCAN(Density-Based Spatial Clustering of Applications with Noise):

DBSCAN is a density-based clustering algorithm that groups data points based on density rather than distance from centroids.

Key Concepts

- **Epsilon (ϵ)**: Defines the radius of a neighborhood around a point.
- **MinPts**: Minimum number of points required to form a dense region.
- **Core Points**: Points with at least **MinPts** neighbors in an ϵ -radius.
- **Border Points**: Points that are reachable from a core point but do not satisfy MinPts.
- **Noise Points**: Points that are neither core nor border points.

Algorithm Steps

1. Select an **unvisited** point x .
2. Retrieve all points in the ϵ -neighborhood of x .
 - If x is a **core point**, form a new cluster.
 - If x is a **border point**, assign it to an existing cluster.
 - If x is a **noise point**, mark it as noise.
3. Expand clusters recursively by including density-reachable points.
4. Repeat until all points are visited.

Mathematical Formulation

A point p is **density-reachable** from q if there exists a sequence of points $p_1, p_2, \dots, p_n$ such that:

$$p_1 = q, \quad p_n = p, \quad \text{and} \quad d(p_i, p_{i+1}) \leq \epsilon$$

for all i , and at least one p_i is a core point.

Advantages of DBSCAN

- Can detect arbitrary shaped clusters.
- Handles noise effectively.
- No need to specify the number of clusters.

Limitations

- Choosing the right ϵ and MinPts is non-trivial.
- Struggles with varying densities.

Evaluation Metrics for Clustering

Since clustering is **unsupervised**, we cannot use traditional classification metrics (like accuracy). Instead, we use:

1. Silhouette Score:

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where:

- $a(i)$ is the average intra-cluster distance.
- $b(i)$ is the average inter-cluster distance.
- $S(i)$ ranges from -1 (incorrect clustering) to 1 (good clustering).

Output:

1] Import the dataset:-

```
df = pd.read_csv(r"C:\Users\adity\Downloads\Battery_RUL.csv")
df.head()
```

	Cycle Index	Discharge Time (s)	Decrement 3.6-3.4V (s)	Max. Voltage Dischar. (V)	Min. Voltage Charg. (V)	Time at 4.15V (s)	Time constant current (s)	Charging time (s)	RUL
0	1.0	2595.30	1151.488500	3.670	3.211	5460.001	6755.01	10777.82	1112
1	2.0	7408.64	1172.512500	4.246	3.220	5508.992	6762.02	10500.35	1111
2	3.0	7393.76	1112.992000	4.249	3.224	5508.993	6762.02	10420.38	1110
3	4.0	7385.50	1080.320667	4.250	3.225	5502.016	6762.02	10322.81	1109
4	6.0	65022.75	29813.487000	4.290	3.398	5480.992	53213.54	56699.65	1107

2] Standardize the data fields:-

```
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
```

3] Plot the elbow curve:-

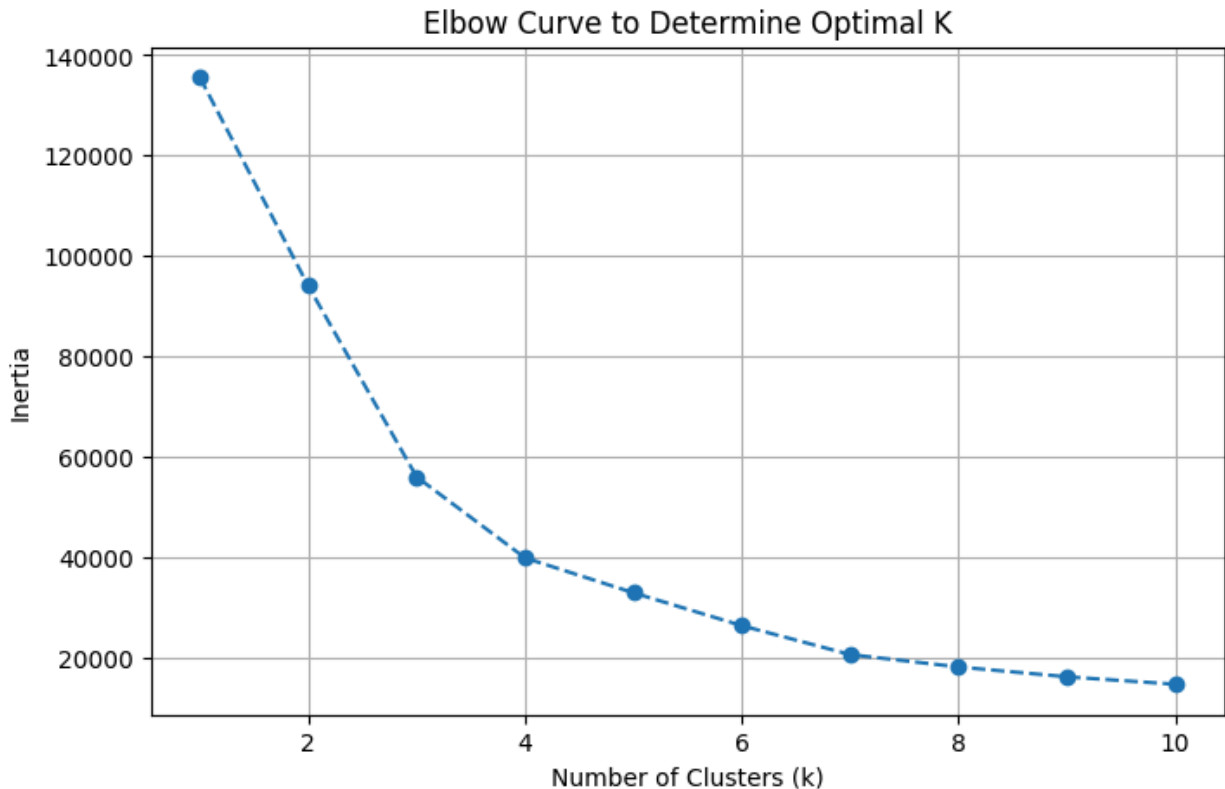
```
inertia = []
k_range = range(1, 11)
```

for k in k_range:

```
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df_scaled)
    inertia.append(kmeans.inertia_)
```

```
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o', linestyle='--')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
```

```
plt.title("Elbow Curve to Determine Optimal K")
plt.grid(True)
plt.show()
```



```
4]Apply the K-means algorithm library and print its silhouette score.
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(df_scaled)
```

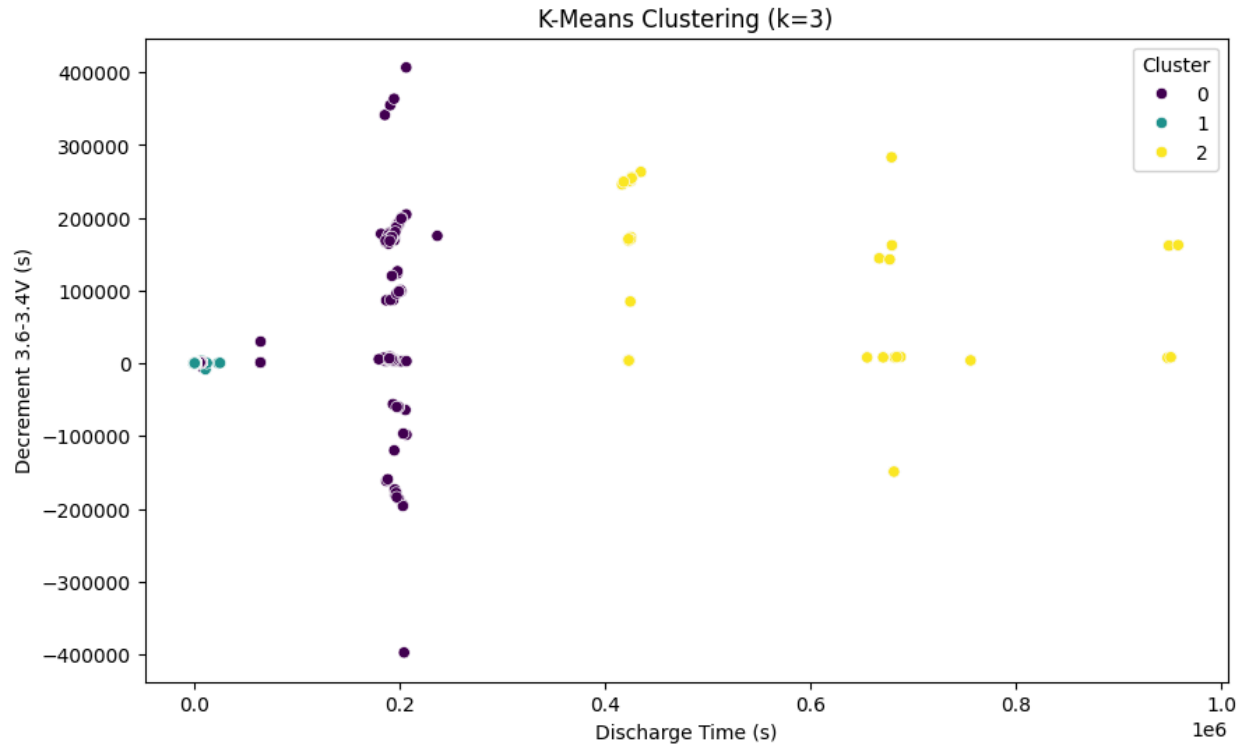
```
sil_score = silhouette_score(df_scaled, df['Cluster'])
print(f"Silhouette Score for k={optimal_k}: {sil_score:.4f}")
```

Silhouette Score for k=3: 0.5352

5]Plot the k-means clustering graph:-

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x=df.iloc[:, 1], y=df.iloc[:, 2], hue=df['Cluster'], palette='viridis')
plt.xlabel(df.columns[1])
plt.ylabel(df.columns[2])
plt.title(f"K-Means Clustering (k={optimal_k})")
plt.show()
```

```
print("Cluster Centers (Scaled):\n", kmeans.cluster_centers_)
```



Cluster Centers (Scaled):

```
[[ -8.55089370e-01  3.24861230e-02  3.07492857e-02  7.25227503e-01
  -7.17684572e-01  1.49146360e-01  3.83739291e-02  7.96357978e-04
   8.55145237e-01]
 [ 8.41716736e-01 -1.01867078e-01 -6.08107745e-02 -7.28209229e-01
   7.18680028e-01 -2.03649250e-01 -1.11423748e-01 -8.31348558e-02
  -8.41773487e-01]
 [-9.94361326e-02  1.76577394e+01  7.71857917e+00  3.70244272e+00
  -3.17003617e+00  1.43739585e+01  1.86084381e+01  2.08019431e+01
   9.98866315e-02]]
```

DBScan:-

1] Plot the k distance graph to get the epsilon value:-
 from sklearn.neighbors import NearestNeighbors

k = 5

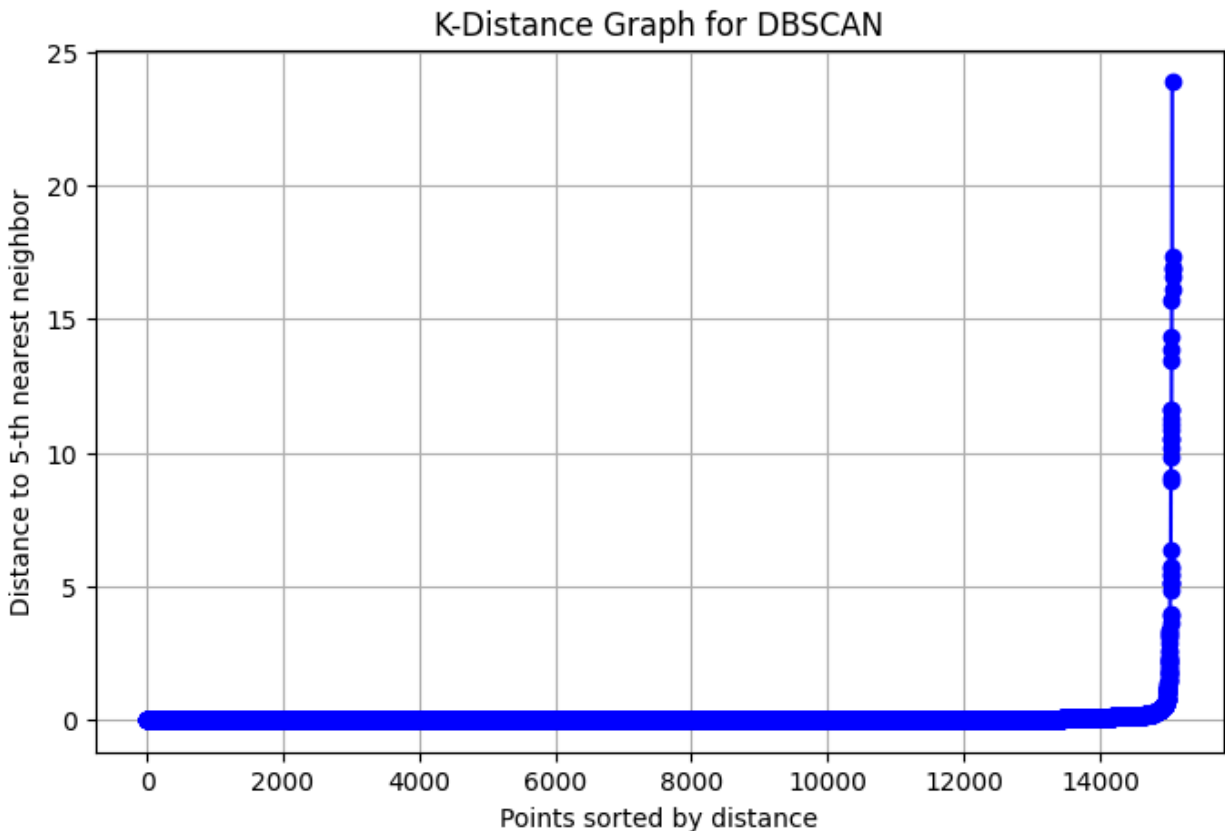
```
nbrs = NearestNeighbors(n_neighbors=k)
nbrs.fit(df_scaled)
```

```
distances, indices = nbrs.kneighbors(df_scaled)
k_distances = np.sort(distances[:, k-1])
```

```

plt.figure(figsize=(8, 5))
plt.plot(range(len(k_distances)), k_distances, marker='o', linestyle='-', color='b')
plt.xlabel("Points sorted by distance")
plt.ylabel("Distance to {k}-th nearest neighbor")
plt.title("K-Distance Graph for DBSCAN")
plt.grid(True)
plt.show()

```



2] Plot the DBScan plot for with the optimal esp value.(5 is the value in our case)

```
from sklearn.cluster import DBSCAN
```

```
eps = 2.5
```

```
min_samples = 5
```

```
dbscan = DBSCAN(eps=eps, min_samples=min_samples)
```

```
df['DBSCAN_Cluster'] = dbscan.fit_predict(df_scaled)
```

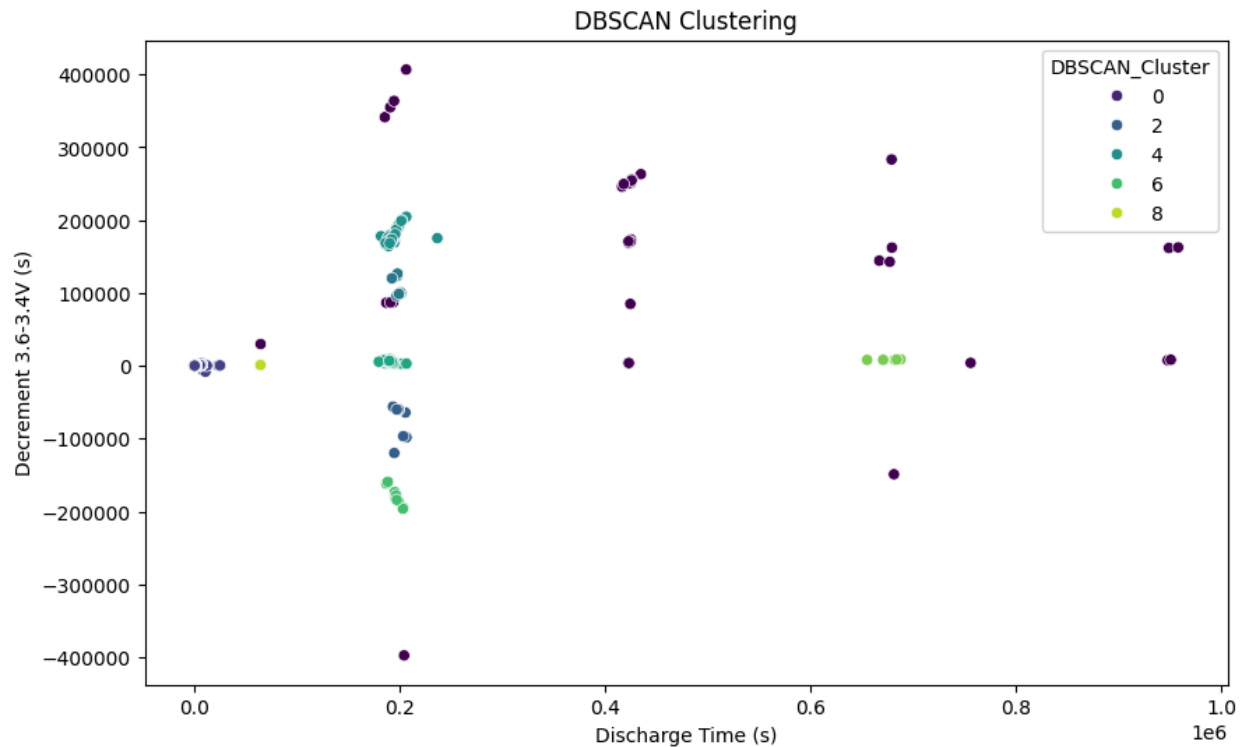
```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(x=df.iloc[:, 1], y=df.iloc[:, 2], hue=df['DBSCAN_Cluster'], palette='viridis')
```

```
plt.xlabel(df.columns[1])
```

```
plt.ylabel(df.columns[2])
```

```
plt.title("DBSCAN Clustering")
plt.show()
```



Conclusion:

The clustering analysis on the **Battery RUL dataset** using K-Means and DBSCAN reveals key insights into the dataset's structure. The **elbow method** suggested an optimal **k=3**, and the **silhouette score of 0.5352** indicates moderate cluster separation, though some overlap is present. This overlap could be attributed to high feature correlation, data distribution issues, or scaling effects. K-Means effectively groups the data into distinct clusters but struggles when clusters are not well-separated. On the other hand, **DBSCAN**, being a density-based algorithm, is better suited for datasets with noise and arbitrary-shaped clusters. However, DBSCAN's performance depends heavily on selecting the right **ϵ (epsilon)** and **MinPts**, which can be challenging. Given the dataset's size (**16,000 rows × 10 columns**), K-Means proves computationally efficient, but **feature selection, dimensionality reduction (PCA), or alternative distance metrics** could further refine clustering quality. Overall, while K-Means provides a structured clustering approach, DBSCAN could be useful if the dataset contains noise or irregularly shaped clusters. Further improvements could include **PCA for dimensionality reduction** and **hyperparameter tuning for DBSCAN** to optimize results.